

Projet : Plateforme web de vente de fichiers STL et service d'impression 3D

Table des matières

Contexte et objectif.....	1
MERN = MongoDB + Express + React + Node.js.....	2
Le MERN amélioré.....	2
Public cible et positionnement.....	3
Valeur ajoutée du projet.....	3
Fonctionnalités.....	4
Partie Utilisateur.....	4
Partie Administrateur.....	5
Architecture technique.....	5
Front-end.....	5
Back-end.....	6
Performances et scalabilité.....	7
Sécurité et sauvegardes.....	7
Conformité RGPD.....	8
Gestion des rôles et droits d'accès.....	8
Hébergement envisagé.....	8
Arborescence complète du site.....	9
Pages principales.....	9
Arborescence du projet.....	9
Schéma MongoDB.....	10
Détails du modèle User.....	10
Diagramme UML et Cas d'utilisation.....	11
Planification et livrables.....	14
Diagramme de Gantt.....	15
Tests :.....	15
Justification du choix de mon architecture :.....	18
1. Vercel (Pour le Front-end React/Vite).....	18
2. Render (Pour le Back-end Node.js/Express).....	18
3. Cloudinary (Pour le stockage des fichiers STL et images).....	19
4. MongoDB Atlas (Pour la Base de données).....	19

Contexte et objectif

Ce projet, c'est la création d'un **site web** où l'on peut **vendre** et **acheter** des fichiers STL et **commander** des **impressions 3D**.

L'idée, c'est de proposer un endroit simple et sympa pour acheter, vendre ou faire imprimer des modèles 3D, facilement.

Le site doit être facile à utiliser, rapide et bien fait, avec un **affichage 3D** en ligne des fichiers grâce à **Three.js**. Le but, c'est de créer une **application** à la fois **pro**, **agréable** et qui peut **évoluer** facilement, que ce soit pour un **particulier** ou pour une **entreprise**.

- La **vente et l'achat** de fichiers STL destinés à l'impression 3D.
- La possibilité pour un utilisateur de **commander une impression 3D personnalisée**, soit à partir des fichiers proposés sur la plateforme, soit à partir de ses propres fichiers STL.
- Une **visualisation interactive** des modèles 3D directement dans le navigateur, grâce à une intégration de *Three.js*.

Le site repose sur une **architecture complète MERN améliorée** :

React + TypeScript + Vite (frontend) et Node.js + Express + MongoDB (backend).

MERN = MongoDB + Express + React + Node.js

Lettre Composant		Rôle
M	MongoDB	Base de données NoSQL, pour stocker les fichiers, utilisateurs, commandes, etc.
E	Express.js	Framework minimal pour Node.js — gère les routes et la logique serveur (API REST).
R	React.js	Framework front-end (interface utilisateur) — permet de construire le site web interactif.
N	Node.js	Environnement JavaScript côté serveur — exécute Express et communique avec MongoDB.

Le MERN amélioré

Amélioration	Description	Ce que ça t'apporte
TypeScript	Ajoute un typage fort à React + Node	Moins d'erreurs, meilleure autocomplétion
Vite (au lieu de Create React App)	Build tool ultra rapide pour le front	Chargement instantané, hot reload
Architecture "modulaire"	/models, /controllers, /routes séparés	Code clair, scalable
Three.js intégré au front	Visualisation 3D directe	Expérience utilisateur moderne
Axios + API REST complète	Communication front ↔ back	Structure professionnelle
dotenv (.env)	Configuration propre (ports, URLs, secrets)	Sécurité et portabilité
Nodemon + ts-node	Serveur auto-reload en dev	Productivité accrue
Design responsive (CSS variables + grid)	Adapté mobile / desktop	UX pro sans frameworks lourds

Public cible et positionnement

La plateforme s'adresse principalement :

- aux **designers 3D indépendants** souhaitant vendre leurs créations STL,
- aux **entreprises** recherchant un service d'impression rapide et fiable,
- et aux **étudiants ou makers** désirant partager ou tester des modèles 3D.

Le positionnement de la plateforme repose sur :

- une **expérience utilisateur fluide et professionnelle**,
- une **infrastructure technique moderne**,
- et une **qualité de rendu des modèles 3D** supérieure grâce à la visualisation intégrée Three.js.

Valeur ajoutée du projet

Contrairement aux plateformes existantes comme [Cults3D](#) ou [Thingiverse](#), cette application vise une approche centrée sur :

- une **visualisation 3D intégrée** des fichiers STL, sans installation de logiciel tiers
- une **interface optimisée** pour la consultation, l'achat et la commande d'impressions 3D personnalisées
- une **infrastructure moderne et performante**, reposant sur un environnement **full-stack JavaScript (MERN amélioré)**
- un **système d'impression sur demande**, permettant d'envoyer son propre fichier STL pour fabrication,
- une **séparation claire des responsabilités** (front-end, API, stockage fichiers et base de données) facilitant la maintenance et la montée en charge
- une **architecture full-stack moderne (MERN améliorée)** garantissant des performances élevées et une maintenance facilitée.
- la possibilité de proposer à terme un **module de tarification automatisée** selon la complexité du modèle et la quantité demandée.

La valeur ajoutée de mon projet par rapport aux plateformes existantes est qu'il existe un **service d'impression 3D à la demande** pour ceux qui font du **modélisme 3D** par exemple mais qu'il n'aurait pas d'**imprimante 3D** ou pas d'**imprimante 3D performante**.

Fonctionnalités

Partie Utilisateur

- **Inscription / Connexion**
 - Gestion des comptes utilisateurs (mots de passe sécurisés, sessions JWT à venir).
- **Consultation du catalogue de fichiers STL**
 - Affichage des fichiers avec titres, catégories, prix et description.
 - Système de recherche par mot-clé et filtres par catégorie.
 - Données chargées dynamiquement depuis la base MongoDB.

	Technologie du Web	
	Rapport Projet	

- **Visualisation 3D en ligne des fichiers STL**
 - Aperçu 3D interactif grâce à **Three.js + STLLoader**.
 - Rotation, zoom et déplacement de la caméra (OrbitControls).
 - Recentrage et échelle automatique du modèle.
- **Téléchargement de fichiers STL achetés**
- **Simulation de paiement**
 - **Le système de paiement sera simulé à l'aide d'une fausse API REST ou d'un environnement sandbox de Stripe, afin de reproduire un scénario d'achat réaliste sans transaction réelle.**
Les commandes seront ainsi enregistrées avec un statut "payée (simulée)" pour les tests de bout en bout.
- **Envoi de son propre fichier STL (Upload)**
 - L'utilisateur peut importer un fichier STL et ajouter des instructions pour demander une impression personnalisée.
 - Le fichier est réceptionné temporairement par le serveur, puis envoyé et stocké de manière sécurisée sur le cloud Cloudinary. L'API renvoie ensuite une URL sécurisée enregistrée dans MongoDB.
- **Suivi des commandes**
Statuts prévus : en attente, en cours, terminé.
- **Espace personnel utilisateur**
 - Historique des achats et téléchargements.
 - Liste des fichiers envoyés pour impression.

Partie Administrateur

- **Gestion des utilisateurs**
(création, modification, suppression — côté back + interface React Admin).
- **Gestion du catalogue STL**
 - Ajout de nouveaux fichiers STL (upload côté serveur).
 - Suppression et modification des fiches existantes.
 - Les fichiers sont automatiquement reliés à la base MongoDB.

- **Gestion des commandes d'impression 3D**
Interface prévue pour le suivi et la mise à jour du statut des commandes.
- **Statistiques**
nombre total de ventes, téléchargements, utilisateurs, fichiers.

Architecture technique

Front-end

- **Technologies** : React + TypeScript + Vite
- **Librairies principales** :
 - three (STLLoader, OrbitControls)
 - react-router-dom
 - axios (pour la communication API)
- **Structure des pages** :
 - Home, Catalogue, ProductDetail, Upload, Profile, Admin
- **Design** : sobre, sombre, responsive, avec une mise en page en grille et composants modulaires.

Back-end

- **Technologies** : Node.js + Express + TypeScript
- **Structure** :

```
backend/  
  src/  
    index.ts  
    routes/  
      fileRoutes.ts  
    controllers/  
      fileController.ts  
  models/  
    File.ts  
  config/
```

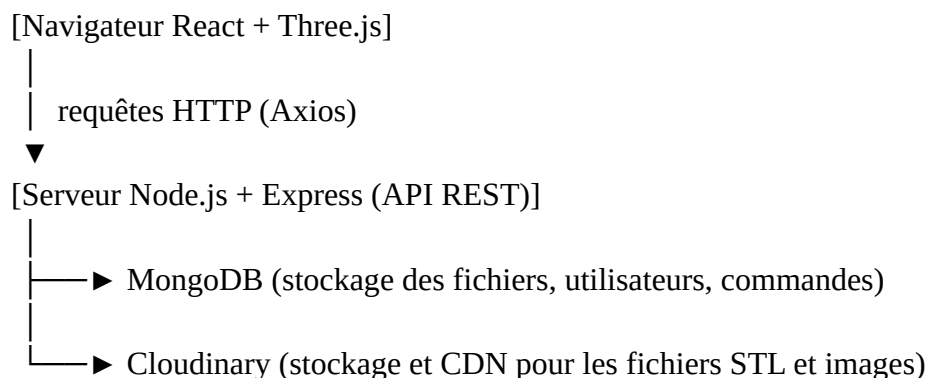
db_mongo.ts

Fonctionnalités back :

- API REST complète /api/files :
 - GET /api/files → liste des fichiers
 - GET /api/files/:title → détail d'un fichier
 - POST /api/files/seed → insertion de données de test
- Intégration de l'API Cloudinary pour l'upload et la distribution des fichiers STL.
- **Base de données** : MongoDB (connexion locale ou Atlas)
- Modèle File :

```
{
  title: String,
  category: String,
  price: Number,
  description: String,
  downloads: Number,
  fileUrl: String,
  imageUrl: String
}
```

Schéma technique de l'architecture MERN améliorée



- Le **front-end React** communique via **Axios** avec l'API Express.

	Technologie du Web	
	Rapport Projet	

- L'API interagit avec MongoDB pour les données et utilise les liens sécurisés générés par Cloudinary pour servir les fichiers STL.
- La configuration (ports, URL, clés) est centralisée dans un fichier `.env`.

Performances et scalabilité

- Le code front-end est optimisé via **Vite** pour un chargement instantané.
- L'API Node.js utilise un système de **cache** et des **requêtes asynchrones non bloquantes**.
- L'architecture MERN permet une **scalabilité horizontale** (ajout de serveurs sans refonte).
- Le stockage MongoDB est compatible avec **Mongo Atlas auto-clusterisé**, assurant la résilience du service.

Sécurité et sauvegardes

- Gestion des mots de passe (prévue avec `bcrypt` et `jsonwebtoken`).
- Validation des formulaires côté **client** (React) et côté **serveur** (Express).
- CORS activé pour la communication entre le front (: 5173) et le back (: 3000).
- Les mots de passe seront **hachés avec bcrypt** avant enregistrement.
- Les échanges front ↔ back se feront uniquement via **requêtes HTTPS** (en production).
- Les requêtes MongoDB sont sécurisées (aucune concaténation directe, utilisation de filtres typés).
- Les fichiers STL sont stockés dans un répertoire protégé, non modifiable depuis le front.
- Une **sauvegarde hebdomadaire automatique** de la base MongoDB sera mise en place (Mongo Atlas Backup).
- Les utilisateurs auront des **droits d'accès différenciés** (User / Admin) pour limiter les actions sensibles.

Conformité RGPD

- Les données personnelles (nom, email, mot de passe) sont stockées de manière sécurisée et chiffrée.
- Les utilisateurs disposent d'un droit de suppression et de rectification de leurs données.
- Aucune donnée sensible (moyen de paiement réel, adresse, etc.) n'est conservée en base.
- Les logs serveurs sont anonymisés et purgés automatiquement après 30 jours.
-

Gestion des rôles et droits d'accès

Le système de permissions distingue deux rôles principaux :

- **Utilisateur** : accès restreint à ses propres fichiers, commandes et profil.
- **Administrateur** : gestion complète du catalogue, des utilisateurs et des commandes.

Chaque opération critique (téléchargement, suppression, modification) est vérifiée par un middleware d'autorisation basé sur **JWT (JSON Web Token)**.

Hébergement envisagé

(à ajouter à la section *Architecture technique — Backend*)

- **Frontend** : déploiement prévu sur **Vercel** ou **Netlify**.
- **Backend** : hébergement sur **Render** ou **Railway.app** (serveur Node.js).
- **Base de données** : **MongoDB Atlas** (cluster cloud gratuit).
- Les fichiers STL et les images miniatures seront hébergés sur le service cloud externe **Cloudinary**, assurant une persistance des données et une livraison rapide grâce à son CDN.

Arborescence complète du site

Pages principales

Page	Description
Accueil	Présentation du service et mise en avant des fichiers populaires
Catalogue	Recherche, filtres et liste des fichiers STL
Détail fichier STL	Visualisation 3D + description + ajout au panier
Impression 3D (Upload)	Envoi d'un STL personnel pour commander une impression sur mesure
Panier / Commande	Simulation d'achat
Connexion / Inscription	Création de compte et authentification sécurisée des utilisateurs
Espace utilisateur	Profil, historique d'achats et suivi des commandes
Administration	Tableau de bord de gestion des fichiers, utilisateurs et commandes

Arborescence du projet

```

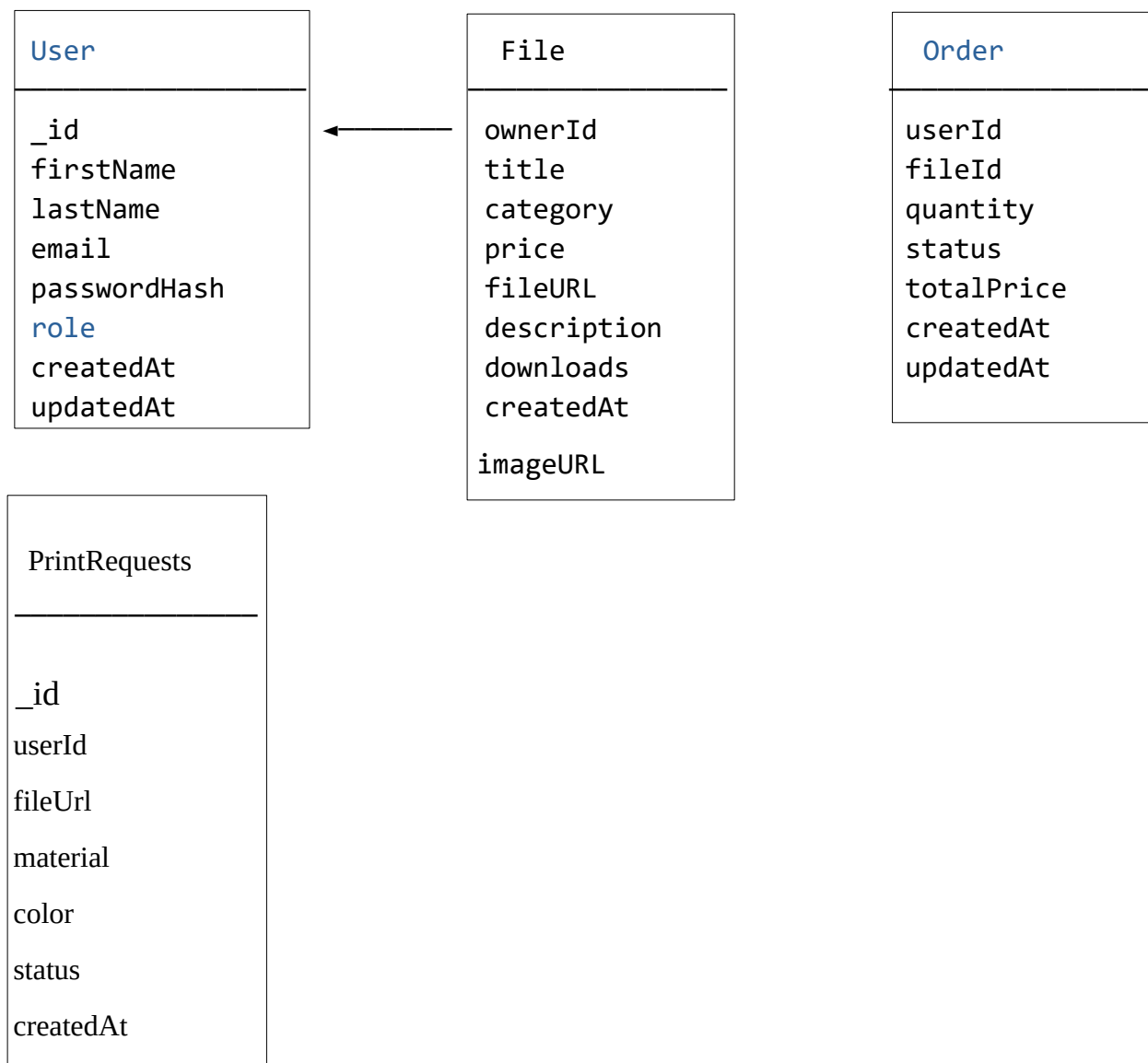
STLMarket/
├── backend/
│   ├── src/
│   │   ├── config/
│   │   │   └── db_mongo.ts
│   │   ├── controllers/
│   │   │   └── {file, order, print, user}Controller.ts
│   │   ├── middlewares/
│   │   │   └── auth.ts
│   │   ├── models/
│   │   │   └── {File, Order, PrintRequest, User}.ts
│   │   ├── routes/
│   │   │   └── {adminPrint, auth, file, order, print}Routes.ts
│   │   └── index.ts

```

```

├── seed.ts
├── package.json
├── .env
├── frontend/
│   ├── src/
│   │   ├── assets/
│   │   │   └── logo.png
│   │   ├── components/
│   │   │   ├── Footer.tsx
│   │   │   ├── Navbar.tsx
│   │   │   └── STLViewer.tsx
│   │   ├── context/
│   │   │   ├── AuthContext.tsx
│   │   │   └── CartContext.tsx
│   │   ├── lib/
│   │   │   └── {api, demo, types}.ts
│   │   └── pages/
│   │       └── {Admin, Cart, Catalogue, Home, Login, PrintService, ProductDetail, Profile,
│   │           Register, Upload}.tsx
│   ├── App.tsx
│   ├── index.css
│   └── main.tsx
├── index.html
├── package.json
├── vite.config.ts
├── .env
├── .gitignore
└── README.md
  
```

Schéma MongoDB



Détails du modèle User

Champ	Type	Description
_id	ObjectId	Identifiant MongoDB unique
firstName	String	Prénom de l'utilisateur
lastName	String	Nom de famille
email	String (unique)	Sert d'identifiant de connexion
passwordHash	String	Mot de passe haché (bcrypt)
role	String ("USER" ou "ADMIN")	Gestion des droits
createdAt, updatedAt	Date	Suivi des inscriptions / modifications

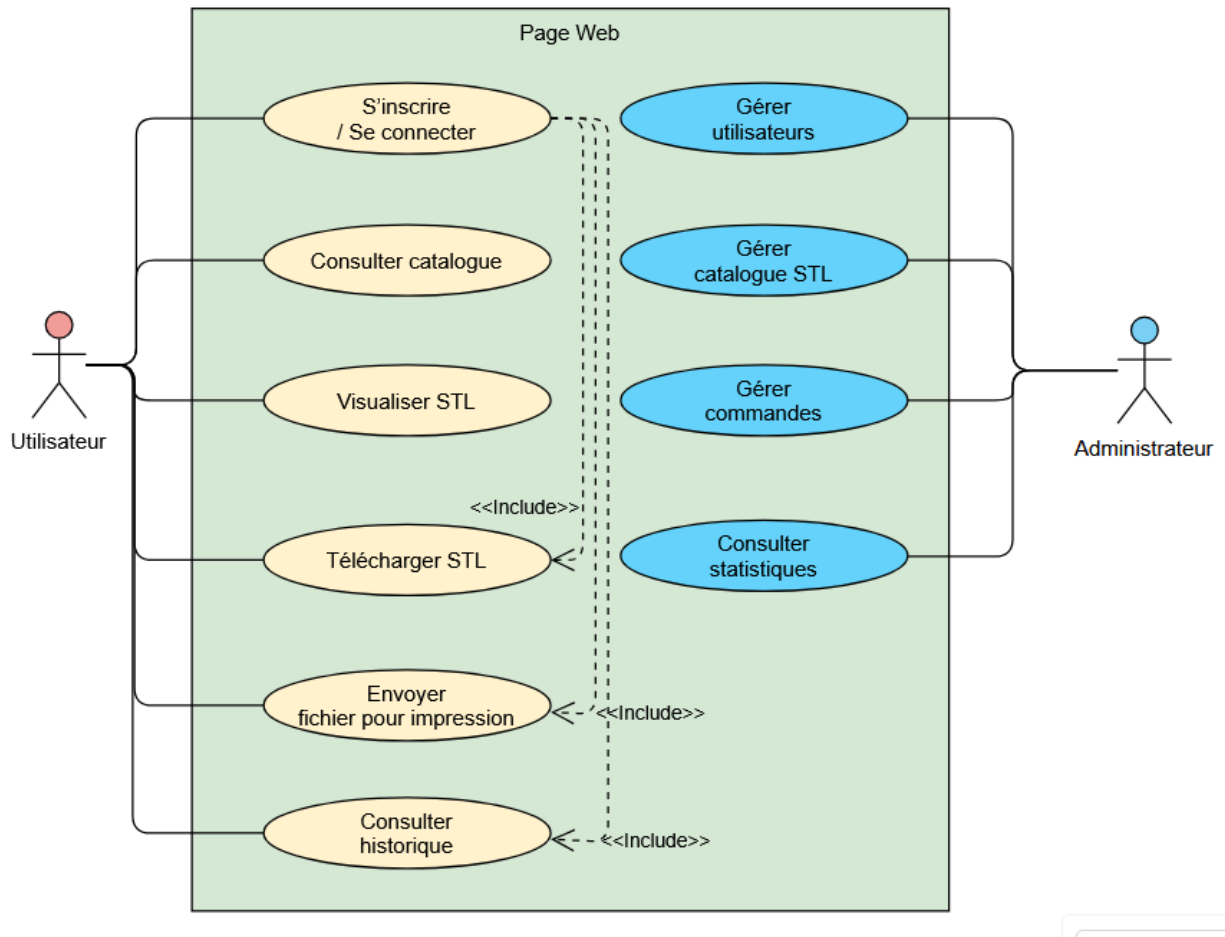
Diagramme UML et Cas d'utilisation

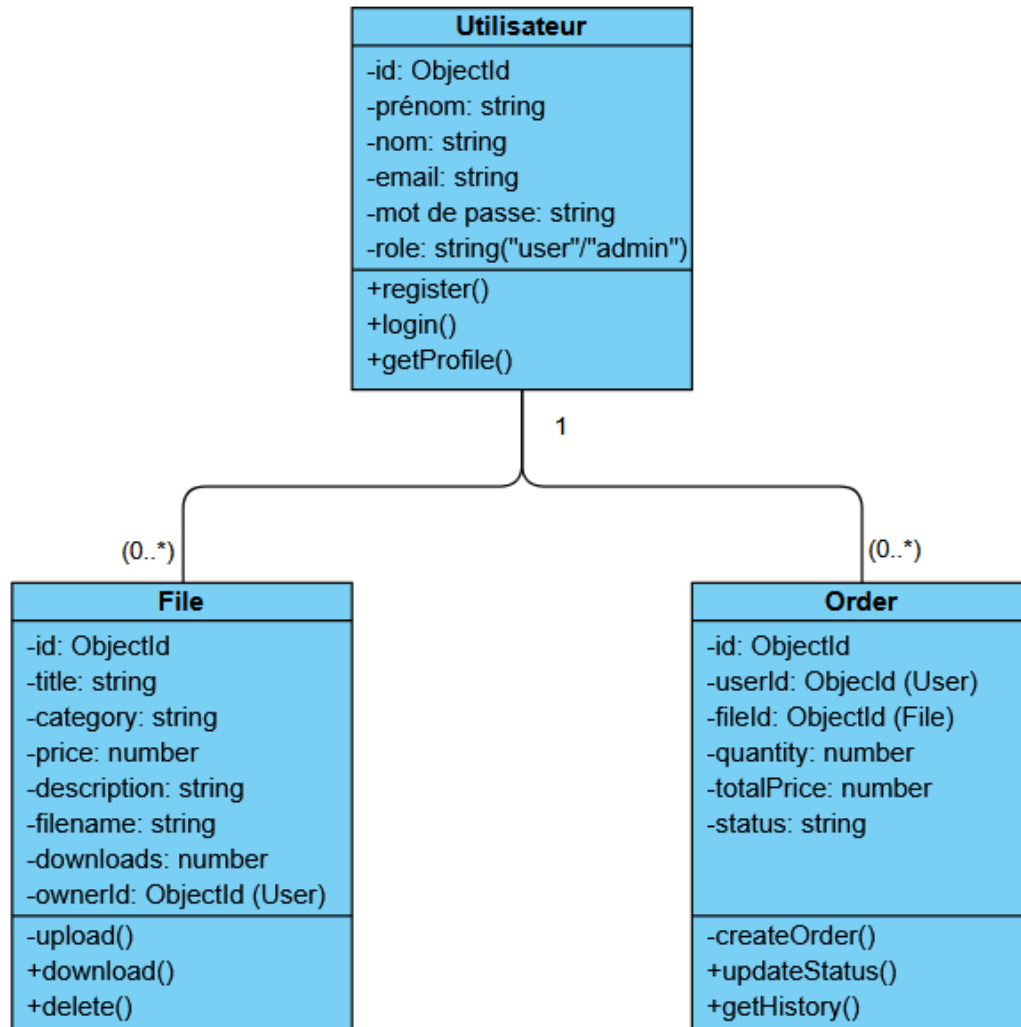
[Utilisateur]

- └─> S'inscrire / Se connecter
- └─> Consulter catalogue
- └─> Visualiser STL
- └─> Ajouter au panier
- └─> Télécharger STL
- └─> Envoyer fichier pour impression
- └─> Consulter historique

[Administrateur]

- └─> Gérer utilisateurs
- └─> Gérer catalogue STL
- └─> Gérer commandes
- └─> Consulter statistiques

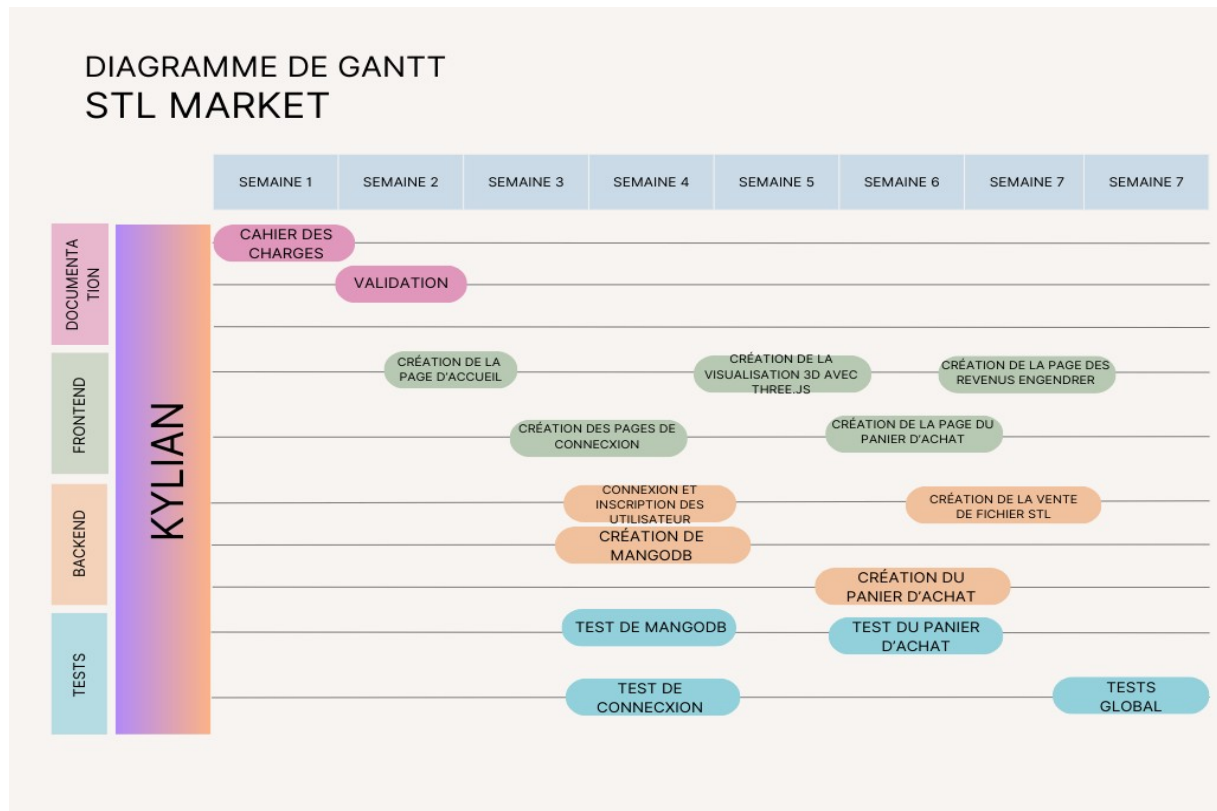




Planification et livrables

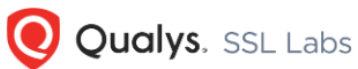
Phase	Période estimée	Objectifs principaux	Livrables	Critères de validation
Phase 1 – Initialisation	Semaine 1 à 3	Création de l'arborescence, configuration Vite/Express, base Mongo connectée	Serveur API + affichage catalogue	API /api/files renvoie les fichiers
Phase 2 – Visualisation 3D	Semaine 4 à 6	Intégration Three.js + affichage STL dynamique	Viewer 3D fonctionnel	Le modèle STL s'affiche correctement
Phase 3 – Upload et Auth	Semaine 7 à 9	Upload STL côté utilisateur + création comptes	Formulaire upload + CRUD fichiers	Les fichiers sont ajoutés et listés
Phase 4 – Paiement simulé et admin	Semaine 10 à 12	Simulation achat + interface admin	Dashboard admin	Toutes les routes CRUD admin fonctionnelles
Phase 5 – Finalisation	Semaine 13	Tests finaux, sécurité, rapport PDF	Documentation finale + rendu	Validation par tests utilisateurs

Diagramme de Gantt



Tests :

SSL Labs :


Home Projects Qualys Free Trial Contact

You are here: [Home](#) > [Projects](#) > [SSL Server Test](#) > stlmarket.onrender.com

SSL Report: stlmarket.onrender.com

Assessed on: Wed, 25 Feb 2026 16:31:43 UTC | [Hide](#) | [Clear cache](#)

[Scan Another >>](#)

	Server	Test time	Grade
1	216.24.57.251 Ready	Wed, 25 Feb 2026 16:29:54 UTC Duration: 66.153 sec	A
2	216.24.57.7 Ready	Wed, 25 Feb 2026 16:31:00 UTC Duration: 43.655 sec	A

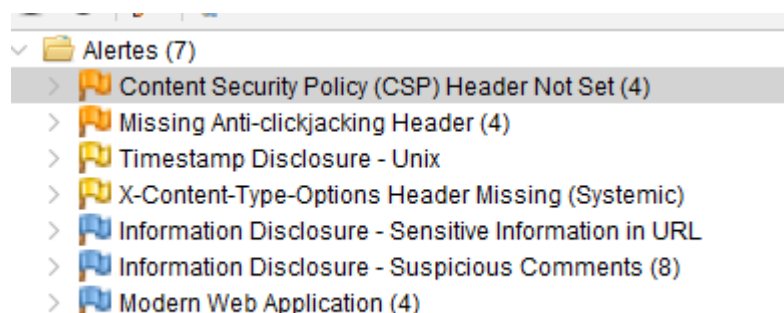
SSL Report v2.4.1

Copyright © 2009-2026 [Qualys, Inc.](#) All Rights Reserved. [Privacy Policy](#).

[Try Qualys for free!](#) Experience the award-winning [Qualys Cloud Platform](#) and the entire collection of [Qualys Cloud Apps](#), including [certificate security](#) solutions.

[Terms and Conditions](#)

OWASP ZAP :



	Technologie du Web	
	Rapport Projet	

Justification du choix de mon architecture :

1. Vercel (Pour le Front-end React/Vite)

Pourquoi ce choix : Vercel est optimisé nativement pour les frameworks basés sur React (comme Vite). Il offre un pipeline **CI/CD (Intégration et Déploiement Continu)** "zéro configuration". Dès qu'un code est poussé sur GitHub, le site est compilé et déployé sur un réseau de distribution mondial (Edge CDN), ce qui garantit un temps de chargement ultra-rapide pour le catalogue 3D.

Pourquoi pas les autres (ex: AWS S3, GitHub Pages ou VPS classique) : Héberger un site React (Single Page Application) sur un VPS classique demande de configurer manuellement le serveur (Nginx/Apache) pour gérer le routage. AWS S3 demande une configuration complexe des certificats SSL et du CDN (CloudFront). Vercel gère tout cela automatiquement en mode PaaS (Platform as a Service).

2. Render (Pour le Back-end Node.js/Express)

Pourquoi ce choix : Render est l'alternative moderne pour héberger des web services. Il se connecte directement à GitHub, gère l'installation des dépendances (NPM) et relance le serveur à chaque mise à jour. C'est une plateforme robuste qui gère nativement le HTTPS et les certificats SSL.

Pourquoi pas les autres (ex: Heroku, AWS EC2) : Heroku a supprimé son offre gratuite et son infrastructure commence à vieillir. Utiliser une machine virtuelle "nue" (comme AWS EC2 ou OVH) aurait nécessité de gérer soi-même l'OS, le pare-feu, les mises à jour de sécurité et les proxys inverses. C'est un travail d'Administrateur Système (SysAdmin) qui dépasse le cadre d'un projet de développement où le but est de prouver le fonctionnement de l'API.

	Technologie du Web	
	Rapport Projet	

3. Cloudinary (Pour le stockage des fichiers STL et images)

Pourquoi ce choix : Les serveurs cloud modernes (comme Render ou Heroku) utilisent des **systèmes de fichiers éphémères**. Si un utilisateur upload un fichier STL directement sur le serveur Render, **ce fichier sera définitivement supprimé** au prochain redémarrage du serveur ! Cloudinary permet de déporter le stockage vers un vrai service cloud persistant. De plus, il agit comme un CDN spécialisé pour délivrer les lourds fichiers 3D très rapidement.

Pourquoi pas les autres (ex: AWS S3) : Bien qu'AWS S3 soit le standard de l'industrie, sa configuration (gestion des rôles IAM, politiques, règles CORS) est excessivement complexe pour un MVP (Minimum Viable Product) étudiant. Cloudinary offre un SDK Node.js très simple et adapté aux délais du projet.

4. MongoDB Atlas (Pour la Base de données)

Pourquoi ce choix : C'est le choix naturel de la stack MERN. Atlas est un service **DBaaS** (Database as a Service) géré directement par les créateurs de MongoDB. Il offre nativement une haute disponibilité, des sauvegardes automatisées, et surtout une sécurité de base forte (gestion des IPs autorisées en liste blanche, chiffrement des données).

Pourquoi pas les autres (ex: MongoDB en local, SQL/PostgreSQL) : Héberger sa propre base MongoDB sur un serveur expose à de très gros risques de sécurité (ex: ransomwares fréquents sur les bases mal configurées) et de perte de données. Quant au SQL, le NoSQL (MongoDB) était beaucoup plus adapté ici pour stocker des catalogues de fichiers 3D avec des métadonnées flexibles (poids, dimensions, matériaux), sans avoir à gérer des migrations de schémas complexes.